Cryptographic Watermarking of AI-Generated Text: Implementing and Attacking HMAC-Based Token Signatures

Sagnik (Nik) Chakrabarti
Georgia State University
schakrabarti3@student.gsu.edu

Adedamola Akapo
Georgia State University
aakapo1@student.gsu.edu

Abstract - As AI-generated text becomes increasingly indistinguishable from human writing, the need for cryptographically verifiable authorship attribution has grown critical. This paper presents DP-WatermarkCLI, a modular post-generation watermarking system that embeds HMAC-SHA256 signatures invisibly into text documents using zero-width Unicode steganography and verifies them through a four-layer composite confidence scoring mechanism. The system operates on real .docx and .txt files and requires no access to the language model that generated the text. We implement and evaluate four attacks, zero-width character stripping, synonym substitution, truncation, and offline paraphrase simulation, across multiple intensities on a five-document corpus. Results demonstrate that steganographic payloads are trivially destroyed by content-preserving attacks while HMAC integrity survives, yielding confidence of 0.7000 post-strip versus 1.0000 at baseline. Content-modifying attacks reduce confidence to approximately 0.27–0.33 while preserving over 84% semantic similarity. A brute-force key-cracking experiment confirms that system security reduces entirely to key confidentiality.

Index Terms - HMAC, steganography, watermarking, AI-generated text, cryptographic attribution, zero-width characters, Unicode, natural language processing.

I. INTRODUCTION

The proliferation of large language models (LLMs) capable of generating fluent, contextually coherent text has created an urgent demand for reliable authorship attribution mechanisms. Existing statistical detectors such as GPTZero rely on probabilistic heuristics that are readily defeated by minor post-generation edits, and provide no mathematical proof of origin [1]. A more principled approach applies cryptographic techniques, specifically keyed hash functions, to embed verifiable, secret-key-dependent signatures directly into text content.

Two broad paradigms have emerged for watermarking LLM output. The first, pioneered by Kirchenbauer et al. [2], operates during generation by biasing token selection toward a key-derived green list. This approach achieves strong statistical guarantees but requires access to the model's generation process, making it inapplicable to closed commercial APIs such as GPT-4 and Claude. The second paradigm, explored in this paper, operates post-generation: a cryptographic signature derived from the full document is embedded through steganographic channels invisible to human readers.

We implement a hybrid system combining HMAC-SHA256 document fingerprinting with zero-width Unicode character embedding. The core insight motivating this combination is that neither mechanism alone is sufficient: steganographic embedding is trivially stripped without altering visible content, while exact HMAC matching requires the original document for comparison. Together, they create a two-signal attribution system with complementary failure modes that degrades gracefully under adversarial conditions.

This paper makes the following contributions:

1) A modular, open-source CLI implementation supporting .docx and .txt files with no LLM access required.
2) A documented three-layer watermark architecture covering module structure, data flow, and implementation details.
3) A four-layer composite verification architecture with weighted confidence scoring and interpretable verdicts.
4) Systematic evaluation of four attack categories across multiple intensities with quantitative metrics.
5) A brute-force key-cracking demonstration confirming the threat model's dependency on key secrecy.

II. RELATED WORK

The problem of detecting AI-generated text has attracted significant research attention since the introduction of transformer-based language models. Devlin et al. [1] demonstrated that models such as BERT could generate text indistinguishable to untrained readers, motivating authorship detection as a security research challenge.

Kirchenbauer et al. [2] proposed the first principled watermarking scheme, partitioning the vocabulary into green and red lists derived from a secret key and biasing generation toward green tokens. Zhao et al. [3] extended this with provable robustness bounds under paraphrase attacks. However, both schemes require controlling the generation process, which is unavailable for closed APIs.

Post-generation steganographic approaches embed information into whitespace patterns, synonym choices, or invisible Unicode characters [4]. These methods are flexible but fragile under Unicode normalization and copy-paste operations. Bellare et al. [5] established the theoretical foundations of HMAC, demonstrating security under the random oracle model, the basis for our Layer 1 authentication. Merhav and Sabbag [6] analyzed optimal watermark embedding under resource constraints, informing our bit-count selection. Athalye et al. [7] developed adversarial attack methodology that informed our substitution and paraphrase simulation designs.

Our work bridges these paradigms by combining steganographic embedding for keyless blind detection with HMAC integrity verification for authenticated attribution. To our knowledge, this is the first paper to systematically evaluate this combination against four attack categories on real-world .docx files.

III. SYSTEM ARCHITECTURE

DP-WatermarkCLI is organized into four modules that form a complete watermarking pipeline: Tokenizer, Embedder, Verifier, and Attack Suite, exposed through a command-line interface (wm.py).

A. Module Structure

The system is structured as a Python package:

```
CSC8224-Watermark-Full/
├── wm.py        ← CLI (sign/scan/strip/batch/crack)
├── main.py      ← experiment runner
└── watermark/
    ├── core/      tokenizer, embedder, verifier,
    │              file_reader
    ├── attacks/   attack_strip, attack_substitution,
    │              attack_truncation, attack_paraphrase
    └── analysis/  metrics, visualize
```

B. Core Module - Tokenizer

tokenizer.py provides four functions. tokenize_words() splits text into tokens using regex \w+|[^\w\s]. tokenize_sentences() segments at punctuation boundaries via lookbehind (?<=[.!?])\s+. ngrams() generates sliding n-tuples. clean_text() strips all zero-width Unicode characters (U+200B, U+200C, U+200D, U+FEFF, U+2060, U+180E) to normalize text before hashing.

C. Core Module - Embedder

embedder.py implements the three-layer watermark. embed() takes text, a secret key K, and embed_bits (default 64) and returns the watermarked text. Layer 1 (Document HMAC) computes:

$$W_doc = HMAC\text{-}SHA256(K, clean(T)) \quad (1)$$

where clean(T) strips zero-width characters from T. Any visible change produces a completely different W_doc. Layer 2 (Steganographic Embedding) encodes the first 64 bits of W_doc: bit 0 → U+200B, bit 1 → U+200C, prefixed by U+2060 as a synchronization marker. The 65-character payload is injected one character per inter-word gap. Layer 3 (Sentence + Bigram Integrity) computes per-sentence HMACs over all N sentences and signatures over the first 20 sliding word bigrams, enabling localized tamper detection.

D. Core Module - Verifier

verifier.py implements VerificationResult and verify(). Four independent checks are combined:

$$C = 0.35 \cdot E_1 + 0.30 \cdot E_2 + 0.20 \cdot E_3 + 0.15 \cdot E_4 \quad (2)$$

$E_1 \in \{0,1\}$ is an exact HMAC match via hmac.compare_digest() for constant-time comparison. $E_2 \in [0,1]$ extracts the steganographic payload by scanning for the U+2060 sync marker and measures bit agreement. $E_3 \in [0,1]$ is the fraction of original sentence HMACs found in the received document. $E_4 \in [0,1]$ is Jaccard token similarity. Verdicts: HIGH CONFIDENCE ($C \geq 0.85$), DEGRADED ($0.50 \leq C < 0.85$), UNCERTAIN ($0.20 \leq C < 0.50$), NOT WATERMARKED ($C < 0.20$).

E. Core Module - File Reader

file_reader.py provides read_file() dispatching to _read_docx() via python-docx, _read_txt() with UTF-8 encoding, and _read_pdf() via pdfplumber. The write-back path in wm.py preserves paragraph structure by iterating over document paragraphs and replacing runs with the watermarked word sequence.

F. CLI Interface (wm.py)

Five subcommands are exposed: sign embeds a watermark and writes an output file; scan verifies in full mode (with original + key) or blind mode (stego-only, no original needed); batch scans a folder producing a CSV report; strip removes all zero-width characters; crack brute-forces a keylist against the HMAC.

G. Attack Module

Each attack module exposes attack(text, ...) → str and attack_sweep(text, rates) → dict. attack_strip.py calls clean_text() directly. attack_substitution.py uses a 35-entry offline synonym dictionary with a fixed random seed. attack_truncation.py supports suffix, prefix, and random modes. attack_paraphrase.py composes sentence shuffling, synonym substitution, and contraction expansion.

H. Analysis Module

metrics.py implements run_experiment() which embeds watermarks, applies each attack at multiple intensities, verifies, and returns structured results. visualize.py generates four matplotlib figures: confidence vs. intensity (line), semantic preservation vs. intensity, confidence-semantic tradeoff scatter, and an attack capability radar chart, saved to figures/ at 150 DPI.

IV. METHODOLOGY

A. Embedding Pipeline

The pipeline proceeds as follows: (1) read_file() extracts raw text; (2) clean_text() strips existing zero-width characters; (3) HMAC-SHA256 is computed with the secret key to produce a 256-bit digest; (4) the first 64 bits are encoded as a zero-width sequence prefixed with U+2060; (5) the payload is injected one character per inter-word gap; (6) the watermarked text is written back preserving .docx paragraph structure.

B. Verification Pipeline

Verification accepts the received document, the original document, and the secret key. The received text is cleaned and its HMAC compared against the original using constant-time comparison to prevent timing attacks. Steganographic bits are recovered by scanning for the sync marker. Sentence-level HMACs are cross-matched. Jaccard similarity is computed. The four scores are combined per equation (2) and a verdict assigned.

C. Attack Design

Attack 1 - ZW Strip: Removes all zero-width Unicode characters in a single O(n) scan. Targets Layer 2 exclusively; no visible content altered.

Attack 2 - Synonym Substitution: Replaces a fraction r of content words with synonyms from a 35-entry offline dictionary. Rate swept over $r \in \{0.05, 0.10, 0.20, 0.30, 0.50, 0.70, 1.0\}$.

Attack 3 - Truncation: Removes the last r fraction of sentences in suffix mode, sweeping $r \in \{0.10, \ldots, 0.70\}$. Directly reduces E_3 and E_4 while breaking E_1.

Attack 4 - Paraphrase Simulation: Combines sentence-order shuffling, synonym substitution at rate r, and contraction expansion (e.g., can't → cannot). Rate swept over $r \in \{0.10, \ldots, 0.60\}$. Targets all four layers simultaneously.

V. EXPERIMENTS

A. Experimental Setup

Experiments were conducted on a corpus of five AI-generated text samples covering NLP, cryptography, federated learning, transformer architectures, and HMAC authentication, ranging from 87 to 2,084 words. The primary evaluation file was a real .docx document of 2,084 words and 57 sentences. All watermarks used embed_bits = 64. Experiments ran in Python 3.12 on Windows 11. Semantic preservation was measured using Jaccard token similarity; sentence-transformer cosine similarity is deferred to future work.

B. Baseline Verification

Without any attack, all five documents achieve C = 1.0000, with $E_1 = 1$, $E_2 = 1.0$ (64/64 bits recovered), $E_3 = 1.0$, and $E_4 = 1.0$. On the primary evaluation file, 65 invisible characters were embedded, 57 sentence HMACs computed, and all 57 verified intact. This confirms correct operation across the full pipeline.

C. Attack Results

Table I summarizes results at representative intensities. The ZW Strip attack yields C = 0.7000, with $E_1 = 1$ (HMAC intact, visible content unchanged) and $E_2 = 0$ (steganographic payload completely destroyed). This demonstrates that the HMAC layer provides a durable fallback attribution signal after the steganographic channel is eliminated.

Synonym substitution at 30% yields C = 0.3316, with 52/57 sentences intact and $E_4 = 0.994$. At 100% substitution, confidence reaches C = 0.178. Truncation at 30% yields C = 0.2722, $E_4 = 0.848$. Paraphrase simulation at 40% yields C = 0.3165. All content-modifying attacks produce $E_2 = 0$ because word-space reordering disrupts sync marker recovery.

TABLE I Watermark Verification Results Under Four Attack Types

Attack	Rate	Conf.	Bits	Sent.	Jaccard	Verdict
None (baseline)	-	1.0000	64/64	57/57	100.0%	HIGH
ZW Strip	100%	0.7000	0/64	57/57	100.0%	DEGRADED
Substitution	30%	0.3316	0/64	52/57	99.4%	UNCERTAIN
Truncation	30%	0.2722	0/64	~40/57	84.8%	UNCERTAIN
Paraphrase	40%	0.3165	0/64	~43/57	~95.0%	UNCERTAIN

Fig. 1 shows confidence decay as a function of attack intensity for the three content-modifying attacks. All three curves are stable below 20% intensity and decay more sharply above 30%. This stability is explained by the weights: E_1 contributes 0.35 and is binary, a single word substitution breaks the document HMAC regardless of how many words change, so the minimum achievable C under any content attack is $E_3 \times 0.20 + E_4 \times 0.15$, yielding the residual 0.27–0.33 observed.

D. Key-Cracking Demonstration

The crack command was executed against a four-entry keylist containing three decoy keys and the correct signing key. The system correctly identified the signing key at confidence 0.8547, consistent with an exact HMAC match and full bit recovery. This confirms that the security of the HMAC layer reduces entirely to key secrecy: an adversary who enumerates a small key space can fully verify and forge watermarks.

E. Semantic Preservation Analysis

All four attacks preserve semantic content while breaking the watermark: synonym substitution at 30% preserves 99.4% Jaccard similarity, paraphrase at 40% preserves ~95%, and truncation at 30% preserves 84.8%. All attacks achieve C < 0.50 while maintaining high readability, validating them as realistic adversarial scenarios against deployed watermarking systems.

VI. DISCUSSION

The results reveal a fundamental asymmetry between the two watermarking channels. The steganographic channel (Layer 2) is trivially eliminated by a content-preserving attack requiring zero cryptographic knowledge, but enables blind detection without any reference document. The HMAC channel (Layer

1) is computationally secure under key secrecy but requires both the original document and the secret key for verification.

This asymmetry motivates a practical two-mode deployment model. In public detection mode, any party scans for the U+2060 sync marker to detect watermark presence without the original or key, useful for institutional plagiarism screening. In authenticated attribution mode, the document owner performs a full HMAC comparison with the secret key to produce legally meaningful proof of authorship. These two modes serve distinct trust levels.

The composite confidence score's graceful degradation has an important interpretation: a score of $C \approx 0.33$ under 30% word substitution does not indicate inauthenticity, but rather that the document appears to be a substantially modified derivative of a watermarked original, precisely the correct verdict in an academic integrity investigation where a student has paraphrased AI-generated text.

A key limitation is that Jaccard token similarity is a surface-form metric that does not capture semantic equivalence. Future work should replace Layer 4 with cosine similarity of sentence-transformer embeddings. The 35-entry synonym dictionary also limits attack coverage; a WordNet-scale expansion would significantly increase substitution attack effectiveness.

VII. CONCLUSION

We presented DP-WatermarkCLI, a post-generation cryptographic watermarking system combining HMAC-SHA256 document fingerprinting with zero-width Unicode steganographic embedding. The system is organized into four documented modules, core, attacks, analysis, and a CLI interface, and operates on real .docx and .txt files without requiring LLM access.

Systematic evaluation of four attacks demonstrates that steganographic payloads are fragile under trivial content-preserving attacks but that HMAC integrity provides a durable fallback attribution signal. Content-modifying attacks degrade confidence to 0.27–0.33 while preserving 85–99% semantic similarity. A brute-force key-cracking demonstration confirms that security reduces to key confidentiality.

Future directions include: (1) replacing Jaccard with sentence-transformer cosine embeddings for E_4; (2) integrating fuzzy hashing (ssdeep/tlsh) for near-duplicate robustness beyond exact HMAC; (3) evaluating real LLM-based paraphrase attacks via commercial APIs; and (4) extending the system with RSA/ECDSA digital signatures for non-repudiable attribution. Source code and interactive demo are available at github.com/sagnik26nik/CryotoProject.

REFERENCES.

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, Minneapolis, MN, 2019, pp. 4171–4186.

[2] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, "A watermark for large language models," in Proc. Int. Conf. Mach. Learn. (ICML), Honolulu, HI, 2023.

[3] H. Zhao, A. Wen, and L. Li, "Provable robust watermarking for AI-generated text," arXiv preprint arXiv:2306.17439, Jun. 2023.

[4] A. Uchendu, T. Le, K. Shu, and D. Lee, "Authorship attribution for neural text generation," in Proc. EMNLP, 2020, pp. 8384–8395.

[5] M. Bellare, R. Canetti, and H. Krawczyk, "Keying hash functions for message authentication," in Proc. CRYPTO, Santa Barbara, CA, 1996, pp. 1–15.

[6] N. Merhav and E. Sabbag, "Optimal watermark embedding and detection strategies under limited detection resources," IEEE Trans. Inf. Theory, vol. 54, no. 1, pp. 255–274, Jan. 2008.

[7] A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok, "Synthesizing robust adversarial examples," in Proc. ICML, Stockholm, 2018, pp. 284–293.